Makefile

do zero ao avançado

Curso prático • Linux (Ubuntu) • GNU Make • Linguagem C • 42 Porto

12 módulos	4 projetos	Nível
De make até Makefiles profissionais	Programa simples, multi-arquivo, libft.a e projeto 42 completo	Iniciante → Avançado

O objetivo é que, ao terminar, você consiga **escrever Makefiles profissionais sem depender de copiar modelos da internet**. O curso é focado em C no Linux e em projetos como a Libft, mas os conceitos servem para C++, Java e qualquer projeto que precise de automação de build.

ATENÇÃO

Nos blocos de código deste PDF, o recuo antes de cada comando de uma regra representa um **TAB real**. Ao digitar no seu editor, use a tecla TAB (e não espaços). Se copiar daqui, troque o recuo por TAB.

Roteiro do curso

Módulo	Conteúdo
1	O que é Makefile e GNU Make
2	Primeira regra e primeiros comandos
3	Targets, prerequisites, recipes e .PHONY
4	Variáveis
5	Compilação de projetos C com vários arquivos
6	Arquivos .o e compilação separada
7	all, clean, fclean e re
8	Bibliotecas estáticas: ar, ranlib e .a
9	Regras de padrão (%.o) e variáveis automáticas ($@, $<, $^)
10	Funções: wildcard, patsubst, addprefix, addsuffix e outras
11	Dependências automáticas com -MMD e -MP
12	Makefile profissional: debug, otimização, erros comuns e boas práticas da 42

Projetos que vamos construir

1. Makefile para um programa C simples (Módulos 2–5).
2. Makefile para um projeto com vários arquivos .c e .h (Módulos 5–7).
3. Makefile profissional para uma biblioteca estática, a **libft.a** (Módulo 8 em diante).
4. Makefile completo seguindo as exigências da 42, usando a libft como dependência (Módulo 12).

Como estudar

- Digite todos os exemplos. Copiar e colar não ensina o Make a você.
- Faça os exercícios de cada módulo antes de ler o gabarito (no final do documento).
- Use `make -n` (simulação) para ver o que o Make faria sem executar nada.
- Quando algo der errado, leia a mensagem de erro inteira: o Make costuma dizer a linha do problema.

MÓDULO 1 — Entendendo o Makefile

1.1 O que é um Makefile?

Imagine um projeto em C com os seguintes arquivos:

```
meu_projeto/
├── main.c
├── utils.c
├── calculadora.c
└── calculadora.h
```

Para compilar manualmente, você poderia executar:

```
gcc main.c utils.c calculadora.c -o programa
```

Até aqui, tudo bem. Mas imagine um projeto com 30, 50 ou 100 arquivos: você teria de digitar todos os nomes a cada compilação. Além disso, ao alterar apenas um arquivo, o ideal é recompilar somente o que realmente mudou.

É aí que entra o **Makefile**: um arquivo de instruções lido pelo programa `make` para automatizar tarefas, principalmente:

- compilar programas;
- gerenciar dependências;
- recompilar apenas os arquivos modificados;
- criar bibliotecas;
- executar testes;
- limpar arquivos temporários;
- automatizar processos de desenvolvimento.

1.2 Instalando o Make no Ubuntu

```
sudo apt update
sudo apt install make build-essential
```

Verifique a instalação:

```
make --version
```

A saída será algo como `GNU Make 4.x`. O pacote `build-essential` também instala ferramentas importantes, como o GCC, o `ar` e o `ld`.

1.3 Criando seu primeiro projeto

```
mkdir curso_makefile
cd curso_makefile
```

Crie um arquivo chamado `main.c`:

main.c

```
#include <stdio.h>

int main(void)
{
    printf("Ola, Makefile!\n");
    return (0);
}
```

Compile manualmente e execute:

```
gcc main.c -o programa
./programa
```

Saída: `Ola, Makefile!`. Você acabou de compilar seu primeiro programa. Agora vamos automatizar isso.

MÓDULO 2 — Criando seu primeiro Makefile

2.1 Criando o arquivo

```
touch Makefile
vim Makefile
```

Escreva:

Makefile

```
programa:
    gcc main.c -o programa
```

Agora execute `make`. O Make imprimirá `gcc main.c -o programa` e criará o executável `programa`.

2.2 Anatomia de uma regra

Um Makefile é composto principalmente por regras no formato:

```
alvo: dependencias
    comando
```

Parte	Nome em inglês	Significado
alvo	target	O que queremos criar ou executar.
dependências	prerequisites	Arquivos ou outros alvos necessários para construir o alvo.
comandos	recipe (receita)	Comandos de shell executados para atingir o alvo.

```
programa: main.c
    gcc main.c -o programa
```

ERRO COMUM
Antes de cada comando da receita, use um **TAB real**, não espaços. Se usar espaços, o Make mostra o erro `*** missing separator. Stop.`

2.3 O que acontece quando executamos make?

O Make procura automaticamente um arquivo de instruções, nesta ordem:

1. `GNUmakefile`
2. `makefile`
3. `Makefile`

Ele usa o primeiro que encontrar. Se você não indicar um alvo, executa o **primeiro alvo do arquivo** (o alvo padrão, ou *default goal*). Para escolher outro alvo, use `make nome_do_alvo`. Para usar um arquivo com outro nome, use `make -f meu_arquivo`.

2.4 Criando mais alvos

Makefile

```
programa:
    gcc main.c -o programa

executar:
    ./programa

limpar:
```

```
	rm -f programa
```

Agora temos três alvos: `make` compila, `make executar` executa e `make limpar` remove o programa.

> **ATENÇÃO**
> Neste exemplo, `make executar` **não** compila o programa se ele ainda não existir. Isso será resolvido no Módulo 3 com dependências entre alvos.

Mini-exercícios

1. Escreva um Makefile com um alvo que compile `main.c` em um executável chamado `ola`.
2. Rode `make -n` e compare a saída com o que o `make` normal faz.
3. Troque o TAB por espaços de propósito e leia a mensagem de erro.

MÓDULO 3 — Targets e dependências

3.1 O que são dependências?

```
programa: main.c
    gcc main.c -o programa
```

Isso significa: *para construir o alvo `programa`, preciso do arquivo `main.c`*. O `main.c` é uma dependência.

Execute `make` duas vezes seguidas. Na segunda vez o Make responde:

```
make: 'programa' is up to date.
```

Isso acontece porque o Make compara as datas de modificação: o arquivo `programa` existe e é **mais novo** que `main.c`, então não precisa ser reconstruído. Se você editar o `main.c` e rodar `make` de novo, o programa será recompilado. Essa é uma das maiores vantagens do Makefile: **evitar compilações desnecessárias**.

3.2 Dependências entre alvos

```
programa: main.c
    gcc main.c -o programa

executar: programa
    ./programa
```

Ao executar `make executar`, o Make:

1. verifica se `programa` existe e está atualizado;
2. se necessário, compila o `main.c`;
3. executa `./programa`.

3.3 Alvos que não são arquivos: .PHONY

```
limpar:
    rm -f programa
```

O alvo `limpar` é só uma tarefa, não um arquivo. Mas, se existir um arquivo chamado `limpar` na pasta, o Make considera o alvo atualizado e **não executa o comando**. Para evitar isso, declaramos o alvo como *phony*:

```
.PHONY: limpar

limpar:
    rm -f programa
```

Isso informa ao Make: *`limpar` não é um arquivo real, execute a receita sempre que for pedido*. Exemplo com vários alvos: `.PHONY: all clean fclean re executar`.

> **DICA**
> Regra prática: todo alvo que **não gera um arquivo com o mesmo nome** deve estar em `.PHONY` (all, clean, fclean, re, run, test...).

Exercícios

1. Crie um arquivo vazio chamado `limpar` e rode `make limpar` sem `.PHONY`. O que acontece?
2. Adicione `.PHONY` e repita o teste.
3. Faça `make executar` funcionar mesmo sem o executável existir.

MÓDULO 4 — Variáveis no Makefile

Variáveis evitam repetição. Compare:

Com repetição

```
programa:
    gcc -Wall -Wextra -Werror main.c -o programa

teste:
    gcc -Wall -Wextra -Werror teste.c -o teste
```

Com variáveis

```
CC = gcc
CFLAGS = -Wall -Wextra -Werror

programa:
    $(CC) $(CFLAGS) main.c -o programa

teste:
    $(CC) $(CFLAGS) teste.c -o teste
```

Item	Significado
`CC`	Compilador (convenção do Make)
`CFLAGS`	Opções de compilação em C (convenção)
`$(CC)`	Usa o valor de CC
`$(CFLAGS)`	Usa o valor de CFLAGS

Significado das flags:

- `-Wall`: habilita vários avisos;
- `-Wextra`: habilita avisos adicionais;
- `-Werror`: transforma avisos em erros.

4.1 Operadores de atribuição

Operador	Funcionamento
`=`	Expansão recursiva: o valor só é calculado quando a variável é **usada**.
`:=`	Expansão imediata: o valor é calculado **na hora** da atribuição.
`?=`	Define somente se a variável ainda não estiver definida (ótimo para valores sobrescritíveis pela linha de comando).
`+=`	Acrescenta conteúdo ao valor atual.

```
CC = gcc
CFLAGS := -Wall -Wextra
CFLAGS += -Werror
NAME ?= programa
```

Resultado: `CC` vale `gcc`, `CFLAGS` vale `-Wall -Wextra -Werror` e `NAME` vale `programa`.

Diferença entre = e :=

```
A = $(B)
B = oi
C := $(B)
D := tchau
```

```
# Aqui: A vale "oi" (resolvido no uso), C vale "" (B ainda não existia
# no momento da atribuição imediata).
```

Sobrescrevendo pela linha de comando

```
make CC=clang
make CFLAGS="-Wall -g"
make NAME=outro_nome     # só funciona porque usamos NAME ?= ou nada impede
```

> **NOTA**
> Variáveis passadas na linha de comando têm prioridade sobre as do Makefile (a menos que você use `override`). Isso é muito útil para trocar compilador ou flags sem editar o arquivo.

Exercícios

1. Crie uma variável `NAME` e use `$(NAME)` como saída do compilador.
2. Experimente `make CC=clang` (se tiver clang instalado) e veja o comando impresso.
3. Descubra, com um `$(info ...)`, o valor de uma variável: `$(info CFLAGS=$(CFLAGS))`.

MÓDULO 5 — Primeiro projeto organizado em C

```
curso_makefile/
├── Makefile
├── main.c
├── matematica.c
└── matematica.h
```

matematica.h

```
#ifndef MATEMATICA_H
# define MATEMATICA_H

int somar(int a, int b);

#endif
```

matematica.c

```
#include "matematica.h"

int somar(int a, int b)
{
    return (a + b);
}
```

main.c

```
#include <stdio.h>
#include "matematica.h"

int main(void)
{
    int resultado;

    resultado = somar(10, 20);
    printf("Resultado: %d\n", resultado);
    return (0);
}
```

Makefile

```
NAME = programa

CC = cc
CFLAGS = -Wall -Wextra -Werror

SRC = main.c matematica.c
HDR = matematica.h

$(NAME): $(SRC) $(HDR)
    $(CC) $(CFLAGS) $(SRC) -o $(NAME)

clean:
    rm -f $(NAME)

.PHONY: clean
```

Execute `make` e depois `./programa`. Saída esperada: `Resultado: 30`.

ATENÇÃO

Correção em relação à versão anterior: o `matematica.h` agora está nas dependências (`HDR`). Sem isso, se você alterasse apenas o header, o `make` diria que está tudo atualizado e **não recompilaria**, deixando o executável desatualizado. Esse problema é resolvido de forma definitiva no Módulo 11.

Este Makefile tem uma limitação: ao mudar **um** arquivo, ele recompila **todos** os `.c`. Os Módulos 6 a 9 resolvem isso com arquivos objeto (`.o`).

Exercícios de fixação (Módulos 1–5)

Crie um Makefile que cumpra os cinco itens abaixo e compare com o gabarito no final do documento.

1. Compilar `main.c` utilizando `cc`, com uma variável `CC`.
2. Criar uma variável `NAME` com o valor `programa` e usar `$(NAME)` como nome do executável.
3. Criar a variável `CFLAGS` com `-Wall -Wextra -Werror` e usá-la no comando de compilação.
4. Criar um alvo `clean` que remova o executável, usando `rm -f` e `.PHONY`.
5. Criar um alvo `executar` que dependa de `$(NAME)`: ao rodar `make executar`, o programa é compilado se necessário e depois executado.

DICA

Tente resolver sozinho antes de olhar o gabarito. Depois, avalie o seu: o que está certo, o que está errado e por quê, como melhorar e como um desenvolvedor experiente escreveria.

MÓDULO 6 — Arquivos .o e compilação separada

6.1 As etapas da compilação

Quando você roda `cc main.c -o programa`, o compilador faz duas coisas seguidas: **compila** cada `.c` em um arquivo objeto (`.o`, código de máquina ainda incompleto) e depois **linka** (liga) todos os objetos em um executável.

```
main.c ──(compilar)──> main.o ──┐
                                ├──(linkar)──> programa
matematica.c ──(compilar)──> matematica.o ──┘
```

Fazendo manualmente, a opção `-c` compila sem linkar:

```
cc -Wall -Wextra -Werror -c main.c -o main.o
cc -Wall -Wextra -Werror -c matematica.c -o matematica.o
cc main.o matematica.o -o programa
```

6.2 Por que separar?

Se você alterar apenas `main.c`, só `main.o` precisa ser refeito; `matematica.o` continua válido. Em projetos grandes isso economiza minutos a cada compilação.

6.3 Makefile com regras explícitas para cada .o

Makefile

```
NAME = programa
CC = cc
CFLAGS = -Wall -Wextra -Werror

$(NAME): main.o matematica.o
    $(CC) main.o matematica.o -o $(NAME)

main.o: main.c matematica.h
    $(CC) $(CFLAGS) -c main.c -o main.o

matematica.o: matematica.c matematica.h
    $(CC) $(CFLAGS) -c matematica.c -o matematica.o

clean:
    rm -f main.o matematica.o

fclean: clean
    rm -f $(NAME)

re: fclean $(NAME)

.PHONY: clean fclean re
```

Faça o teste:

1. `make` → compila tudo.
2. `make` de novo → `up to date`.
3. `touch main.c` e `make` → só `main.o` é refeito e o executável é religado.
4. `touch matematica.h` e `make` → os dois `.o` são refeitos (ambos dependem do header).

> **NOTA**
> Note que o link usa `$(CC)` **sem** `$(CFLAGS)`. As flags de aviso (`-Wall`...) importam na compilação; no link, flags úteis são as de bibliotecas (`-L`, `-l`) e, às vezes, `-fsanitize`.

> **DICA**
> Essa versão funciona, mas é repetitiva: uma regra por arquivo. Nos Módulos 9 e 10 vamos eliminá-la com regras de padrão e funções.

Exercícios

1. Adicione um arquivo `subtrair.c` ao projeto e atualize o Makefile.
2. Use `make -n` após `touch` em cada arquivo e explique o que será recompilado.
3. Remova `matematica.h` das dependências de `main.o` e veja o que acontece ao editar o header.

MÓDULO 7 — all, clean, fclean e re

Quase todo projeto em C usa um conjunto padrão de alvos. Na 42 eles são **obrigatórios**.

Alvo	O que faz
`all`	Constrói o projeto. Deve ser o **primeiro** alvo do arquivo (alvo padrão).
`$(NAME)`	Gera o executável ou a biblioteca.
`clean`	Remove os arquivos objeto (`.o`) e temporários.
`fclean`	Faz `clean` e remove também o executável/biblioteca final.
`re`	Recompila tudo do zero: `fclean` seguido de `all`.

Makefile padrão

```
NAME = programa
CC = cc
CFLAGS = -Wall -Wextra -Werror

SRC = main.c matematica.c
OBJ = $(SRC:.c=.o)

all: $(NAME)

$(NAME): $(OBJ)
    $(CC) $(OBJ) -o $(NAME)

%.o: %.c matematica.h
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(OBJ)

fclean: clean
    rm -f $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

Pontos importantes:

- `all` vem primeiro para que `make` sem argumentos construa o projeto.
- `all: $(NAME)` apenas delega; quem faz o trabalho é a regra de `$(NAME)`.
- `fclean: clean` reaproveita a regra anterior: o Make executa as dependências **antes** da receita.
- `re: fclean all` executa `fclean` e depois `all`, nessa ordem.
- `$(SRC:.c=.o)` troca a extensão de cada nome da lista (substituição de referência), gerando `main.o matematica.o`.

ATENÇÃO
Com `make -j` (execução paralela), as dependências de `re` podem rodar **ao mesmo tempo** e o `all` pode começar antes de o `fclean` terminar. Se isso importar, escreva `re:` com receita: `$(MAKE) fclean` seguido de `$(MAKE) all`. Para projetos 42, `re: fclean all` é o padrão aceito.

7.1 Evitando relink

Na avaliação da 42, rodar `make` duas vezes seguidas **não pode** recompilar nem religar nada. Isso acontece automaticamente quando as dependências estão corretas: o executável depende dos `.o`,

e cada `.o` depende do seu `.c` (e dos headers). Um erro típico que causa relink é declarar um alvo real como `.PHONY` ou fazer o alvo depender de algo que sempre muda.

Exercícios

1. Teste: rode `make` duas vezes. A segunda deve dizer `Nothing to be done for 'all'`.
2. Rode `make re` e observe a ordem dos comandos impressos.
3. Coloque `$(NAME)` em `.PHONY` e veja por que isso causa relink.

MÓDULO 8 — Bibliotecas estáticas: ar, ranlib e .a

8.1 O que é uma biblioteca estática?

Uma biblioteca estática (`.a`, de *archive*) é um pacote de arquivos objeto. Quando um programa usa a biblioteca, o linker copia para o executável apenas o código das funções necessárias. A **libft.a**, da 42, é exatamente isso.

8.2 O comando ar

```
ar rcs libft.a ft_isalpha.o ft_strlen.o ft_atoi.o
```

Opção	Significado
`r`	Insere ou substitui os arquivos dentro do arquivo `.a`.
`c`	Cria a biblioteca se ela não existir (sem avisos).
`s`	Gera o índice de símbolos (equivale a rodar `ranlib`).

`ranlib libft.a` cria o índice separadamente. Como `ar rcs` já faz isso, não é preciso chamar `ranlib` depois, embora seja permitido. Para inspecionar:

```
ar t libft.a        # lista os objetos dentro da biblioteca
nm libft.a          # lista os símbolos (funções) de cada objeto
```

8.3 Makefile da libft

Makefile da libft

```
NAME = libft.a
CC = cc
CFLAGS = -Wall -Wextra -Werror
AR = ar
ARFLAGS = rcs

SRC = ft_isalpha.c ft_isdigit.c ft_strlen.c ft_atoi.c ft_memset.c
OBJ = $(SRC:.c=.o)

all: $(NAME)

$(NAME): $(OBJ)
    $(AR) $(ARFLAGS) $@ $^

%.o: %.c libft.h
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(OBJ)

fclean: clean
    rm -f $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

Aqui usamos duas variáveis automáticas: `$@` (o alvo, `libft.a`) e `$^` (todas as dependências, os `.o`). O Módulo 9 explica todas em detalhe.

8.4 Usando a biblioteca em outro programa

```
cc main.c -L. -lft -o programa          # biblioteca na pasta atual
cc main.c -I./includes -L./libft -lft -o programa
```

Flag	Significado
`-I dir`	Onde procurar headers (`.h`).
`-L dir`	Onde procurar bibliotecas.
`-lft`	Linka com `libft.a` (o prefixo `lib` e a extensão `.a` ficam implícitos).

ERRO COMUM
A **ordem** importa no link: os arquivos `.c`/`.o` vêm antes de `-lft`. Se colocar `-lft` antes, o linker pode dizer `undefined reference`.

8.5 A regra bonus sem relink

Na libft, `make bonus` adiciona funções extras à mesma biblioteca. Uma forma segura de não religar sem necessidade é usar um arquivo-marcador:

```
BONUS_SRC = ft_lstnew_bonus.c ft_lstadd_front_bonus.c
BONUS_OBJ = $(BONUS_SRC:.c=.o)

bonus: .bonus

.bonus: $(OBJ) $(BONUS_OBJ)
    $(AR) $(ARFLAGS) $(NAME) $^
    @touch .bonus

fclean: clean
    rm -f $(NAME) .bonus

.PHONY: all clean fclean re bonus
```

Na primeira execução, `.bonus` não existe, então o `ar` roda e o marcador é criado. Nas seguintes, o Make vê que `.bonus` é mais novo que todos os objetos e não faz nada. O `clean` precisa remover também `$(BONUS_OBJ)`: inclua `rm -f $(OBJ) $(BONUS_OBJ)` na regra `clean`.

Exercícios

1. Monte uma `libmath.a` com `soma.c` e `sub.c` e use-a em um `main.c` com `-L. -lmath`.
2. Use `ar t` e `nm` para listar o conteúdo da biblioteca.
3. Provoque o erro `undefined reference` trocando a ordem de `-l` e explique por quê.

MÓDULO 9 — Regras de padrão e variáveis automáticas

9.1 Regra de padrão (pattern rule)

No Módulo 6 escrevemos uma regra para cada `.o`. Uma **regra de padrão** descreve como transformar *qualquer* `.c` em `.o`:

```
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@
```

O `%` casa com qualquer sequência de caracteres (o *stem*). Para construir `main.o`, o Make procura `main.c`; para `utils.o`, procura `utils.c`.

9.2 Variáveis automáticas

Dentro de uma receita, o Make preenche variáveis especiais com informações da regra em execução:

Variável	Significado	Exemplo em `main.o: main.c util.h`
`$@`	O alvo da regra	`main.o`
`$<`	A **primeira** dependência	`main.c`
`$^`	**Todas** as dependências (sem duplicatas)	`main.c util.h`
`$?`	Dependências **mais novas** que o alvo	as que mudaram
`$*`	O *stem* casado por `%`	`main`
`$(@D)`	Diretório do alvo	`.` (ou `obj/parser` em `obj/parser/a.o`)
`$(@F)`	Nome do arquivo do alvo	`main.o`

> **ATENÇÃO**
> Em regras de padrão como `%.o: %.c foo.h`, o `$<` é sempre o `.c` (primeira dependência). Por isso é comum colocar o `.c` primeiro e os headers depois, e **nunca** usar `$^` na compilação de um `.o` (ele incluiria o `.h` na linha do `cc`).

9.3 Substituição de referência

```
SRC = main.c util.c parser.c
OBJ = $(SRC:.c=.o)        # main.o util.o parser.o
```

A forma `$(VAR:antigo=novo)` troca o sufixo de cada palavra. Para padrões mais gerais, use `$(patsubst %.c,%.o,$(SRC))` (Módulo 10).

9.4 Regra de padrão estática

```
$(OBJ): %.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@
```

Aplica o padrão **somente** aos alvos listados em `$(OBJ)`. É mais restrita e mais segura do que `%.o: %.c` global.

9.5 Separando fontes e objetos em pastas

```
SRC_DIR = src
OBJ_DIR = obj
```

```
SRC = main.c util.c parser/parse.c
OBJ = $(addprefix $(OBJ_DIR)/, $(SRC:.c=.o))

$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c
    @mkdir -p $(@D)
    $(CC) $(CFLAGS) -c $< -o $@
```

O `mkdir -p $(@D)` cria, antes de compilar, a pasta onde o `.o` vai ficar (inclusive subpastas como `obj/parser`). Alternativa mais elegante: uma dependência *order-only*, que vem depois do `|`:

```
$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c | $(OBJ_DIR)
    $(CC) $(CFLAGS) -c $< -o $@

$(OBJ_DIR):
    mkdir -p $@
```

Dependências *order-only* precisam existir antes da receita, mas **não** provocam recompilação quando mudam (a data de uma pasta muda toda vez que se cria um arquivo nela). Essa versão só cria uma pasta de nível único.

9.6 Regras implícitas do Make

O Make já conhece como compilar `.c` em `.o` sem que você escreva nada. A regra embutida equivale a:

```
%.o: %.c
    $(CC) $(CPPFLAGS) $(CFLAGS) -c $< -o $@
```

- `CPPFLAGS` é o lugar certo para `-I` e `-D` (opções do pré-processador).
- `make -p` imprime todas as regras e variáveis conhecidas; `make -r` desativa as regras embutidas.
- Ainda assim, **escreva a sua própria regra**: ela deixa claro o que acontece e a 42 espera isso.

Exercícios

1. Reescreva o Makefile do Módulo 7 usando uma regra estática.
2. Mova os `.c` para `src/` e os `.o` para `obj/`.
3. Compile a regra com `-n` e identifique o valor de `$@`, `$<` e `$^` em cada linha.

MÓDULO 10 — Funções: wildcard, patsubst, addprefix e outras

O Make tem funções de texto embutidas, chamadas com `$(função argumentos)`. Os argumentos são separados por vírgula; **evite espaços** depois das vírgulas quando eles fariam parte do valor.

10.1 Funções de texto

Função	Exemplo	Resultado
`subst`	`$(subst a,o,casa)`	`coso`
`patsubst`	`$(patsubst %.c,%.o,a.c b.c)`	`a.o b.o`
`strip`	`$(strip a b )`	`a b`
`filter`	`$(filter %.c,a.c b.h c.c)`	`a.c c.c`
`filter-out`	`$(filter-out main.c,main.c a.c)`	`a.c`
`sort`	`$(sort b a b c)`	`a b c` (ordena e remove duplicatas)
`words`	`$(words a b c)`	`3`
`word`	`$(word 2,a b c)`	`b`
`firstword`	`$(firstword a b c)`	`a`

10.2 Funções de nomes de arquivos

Função	Exemplo	Resultado
`dir`	`$(dir src/a.c lib/b.c)`	`src/ lib/`
`notdir`	`$(notdir src/a.c lib/b.c)`	`a.c b.c`
`basename`	`$(basename src/a.c)`	`src/a`
`suffix`	`$(suffix src/a.c)`	`.c`
`addprefix`	`$(addprefix obj/,a.o b.o)`	`obj/a.o obj/b.o`
`addsuffix`	`$(addsuffix .c,a b)`	`a.c b.c`
`wildcard`	`$(wildcard src/*.c)`	todos os `.c` de `src/`

10.3 wildcard e a 42

```
SRC = $(wildcard src/*.c)
OBJ = $(patsubst src/%.c,obj/%.o,$(SRC))
```

O `wildcard` descobre os arquivos sozinho, o que é cômodo, mas tem desvantagens: se você adicionar um arquivo por engano, ele entra no build sem você perceber; e um `.c` novo não aparece até o Makefile ser relido.

> **ATENÇÃO**
> Na maioria dos projetos da 42 o uso de `wildcard` (e de `*.c` nas listas) **é proibido pelo avaliador**: espera-se uma lista explícita dos arquivos. Confira o subject de cada projeto. Em projetos pessoais, `wildcard` é aceitável.

10.4 foreach, if, call e shell

```
DIRS = includes libft
INCS = $(foreach d,$(DIRS),-I$(d))        # -Iincludes -Ilibft

OS := $(shell uname -s)                   # Linux, Darwin...
LIBS = $(if $(filter Linux,$(OS)),-lbsd,)  # exemplo de condicional

saudar = echo "Ola, $(1)!"               # variavel parametrizada

hello:
    @$(call saudar,mundo)
```

- `foreach` repete uma expressão para cada palavra de uma lista.
- `if` devolve o segundo argumento se o primeiro não for vazio; senão, o terceiro.
- `call` executa uma variável parametrizada (`$(1)`, `$(2)`...).
- `shell` executa um comando e devolve a saída. Use com moderação: roda sempre que a variável é expandida (prefira `:=`).

10.5 Mensagens: info, warning e error

```
$(info Compilando com $(CC))          # imprime e continua
$(warning Variavel X vazia)            # imprime com o arquivo e a linha
$(error Falta definir NAME)            # imprime e PARA o make
```

> **DICA**
> Essas funções são a melhor ferramenta de *debug* de Makefile: imprima variáveis para ver o que o Make realmente enxerga.

Exercícios

1. Dado `SRC = a.c b.c c.c`, produza `obj/a.o obj/b.o obj/c.o` de duas formas diferentes.
2. Gere `-Ia -Ib -Ic` a partir de `a b c` usando `foreach`.
3. Use `filter-out` para remover `main.c` de uma lista de fontes (útil em testes).

MÓDULO 11 — Dependências automáticas com -MMD e -MP

11.1 O problema

Até aqui, nós listamos os headers nas dependências **manualmente** (`main.o: main.c matematica.h`). Isso falha rápido: um `.c` que inclui três headers, que por sua vez incluem outros... Se você esquecer um, altera o header e o Make não recompila: o programa fica com código antigo e bugs misteriosos.

11.2 A solução: deixar o compilador listar as dependências

O GCC/Clang sabem exatamente quais headers cada `.c` inclui. Com `-MMD`, enquanto compilam, eles escrevem um arquivo `.d` contendo uma regra do Make:

```
cc -MMD -MP -c main.c -o obj/main.o
cat obj/main.d
```

obj/main.d

```
obj/main.o: main.c includes/app.h includes/utils.h

includes/app.h:

includes/utils.h:
```

Flag	Significado
`-MMD`	Gera o `.d` com as dependências, **ignorando** headers de sistema (`<stdio.h>`...). `-MD` incluiria todos.
`-MP`	Adiciona um alvo vazio para cada header. Evita o erro `No rule to make target 'x.h'` quando você apaga ou renomeia um header.
`-MF arq`	(Opcional) escolhe o nome do `.d`.
`-MT alvo`	(Opcional) escolhe o nome do alvo escrito no `.d`.

11.3 Incluindo os .d no Makefile

```
SRC_DIR = src
OBJ_DIR = obj
SRC = main.c util.c
OBJ = $(addprefix $(OBJ_DIR)/, $(SRC:.c=.o))
DEP = $(OBJ:.o=.d)

CPPFLAGS = -I includes

$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c
    @mkdir -p $(@D)
    $(CC) $(CFLAGS) $(CPPFLAGS) -MMD -MP -c $< -o $@

-include $(DEP)
```

- `include` insere o conteúdo de outro arquivo no Makefile, como se tivesse sido digitado ali.
- O hífen em `-include` faz o Make **ignorar** arquivos inexistentes (na primeira compilação, ainda não há `.d`).
- A mesma regra que compila também gera o `.d`; na próxima execução, o Make lê as dependências reais.

ERRO COMUM
Coloque `-include $(DEP)` **no fim** do Makefile (ou depois de `all`). Se vier antes do primeiro alvo, a primeira regra dos `.d` vira o alvo padrão e `make` sem argumentos fará a coisa errada.

11.4 Testando

1. `make` → compila tudo e cria os `.d`.
2. `make` → `Nothing to be done`.
3. `touch includes/utils.h && make` → só os `.c` que incluem `utils.h` são recompilados.
4. Apague `includes/utils.h` e recrie → nenhum erro graças ao `-MP`.

11.5 Limpeza

Lembre-se de remover os `.d` no `clean`. Se os objetos ficam em `obj/`, basta `rm -rf obj`. Se ficam ao lado dos fontes, use `rm -f $(OBJ) $(DEP)`.

NOTA
Alguns subjects da 42 só permitem listar os headers explicitamente e não aceitam `-MMD`. Se for o caso, mantenha a lista manual (`HDR`), como no Módulo 5, e use-a nas dependências de cada `.o`.

Exercícios

1. Aplique `-MMD -MP` ao projeto do Módulo 9 e confira um `.d` gerado.
2. Prove com `touch` que alterar um header recompila só quem o inclui.
3. Comente `-include $(DEP)` e repita o teste. O que muda?

MÓDULO 12 — Makefile profissional

Neste módulo juntamos tudo: organização, build silencioso ou verboso, modo debug, mensagens, depuração do próprio Makefile, erros comuns e as exigências da 42.

12.1 Organização do arquivo

Um Makefile profissional segue sempre a mesma ordem, o que o torna fácil de ler:

1. **Nome e ferramentas:** `NAME`, `CC`, `AR`.
2. **Flags:** `CFLAGS`, `CPPFLAGS`, `LDFLAGS`, `LDLIBS`.
3. **Pastas, fontes e objetos:** `SRC_DIR`, `OBJ_DIR`, `SRC`, `OBJ`, `DEP`.
4. **Alvo padrão** (`all`) e a regra do `$(NAME)`.
5. **Regras de padrão** (`%.o`).
6. **Limpeza:** `clean`, `fclean`, `re`.
7. `.PHONY` e `-include $(DEP)` no final.

12.2 Build silencioso ou verboso

Um `@` no início de uma linha esconde o comando. Para alternar entre os modos com `make V=1`:

```
ifeq ($(V),1)
Q :=
else
Q := @
endif

%.o: %.c
    $(Q)$(CC) $(CFLAGS) -c $< -o $@
```

12.3 Mensagens coloridas

```
GREEN := \033[0;32m
RESET := \033[0m

$(NAME): $(OBJ)
    $(Q)$(CC) $(CFLAGS) $(OBJ) -o $@
    @printf "$(GREEN)[OK]$(RESET) %s criado\n" $(NAME)
```

12.4 Modo debug e sanitizers

```
ifdef DEBUG
CFLAGS += -g3 -fsanitize=address,undefined
endif

# uso:  make re DEBUG=1
```

Outra opção é um alvo próprio com variáveis específicas de alvo:

```
debug: CFLAGS += -g3 -O0
debug: re
```

> **ATENÇÃO**
> O Make **não** sabe que as flags mudaram: ele só compara datas dos arquivos. Ao trocar entre debug e normal, rode `make re` (ou `make fclean`) para recompilar tudo com as flags novas. Lembre de usar `$(CFLAGS)` também na linha de link quando usar `-fsanitize`.

12.5 Compilação paralela e otimização

- `make -j4` roda até 4 receitas ao mesmo tempo (`make -j$(nproc)` usa todos os núcleos).

- `-O2` é um bom nível de otimização para release; `-O0 -g3` para debug. Verifique se o subject do projeto permite flags extras.
- Evite `$(shell ...)` em excesso: ele roda toda vez que a variável é expandida.

12.6 Depurando o Makefile

Comando	Uso
`make -n`	Simula: mostra o que seria executado, sem executar.
`make -B`	Força a reconstrução de tudo, ignorando as datas.
`make -d` / `--debug=b`	Explica por que o Make decide refazer (ou não) cada alvo.
`make -p`	Imprime variáveis e regras (inclusive embutidas).
`make -W arq`	Finge que `arq` foi modificado (teste de dependências).
`make -C dir`	Executa o make dentro de outra pasta.
`make VAR=valor`	Sobrescreve variáveis pela linha de comando.
`$(info ...)`	Imprime valores durante a leitura do Makefile.

Truque útil: um alvo que imprime qualquer variável.

```
print-%:
    @echo '$* = $($*)'

# make print-SRC   ->   SRC = main.c util.c
```

12.7 Erros comuns e como resolver

Mensagem	Causa provável / solução
`missing separator. Stop.`	A receita usa espaços em vez de **TAB**, ou falta o `:` em uma regra.
`No rule to make target 'x.c', needed by 'x.o'`	O arquivo `x.c` não existe, está com nome errado ou no caminho errado. Confira `SRC` e `SRC_DIR`.
`Nothing to be done for 'all'`	Está tudo atualizado. Normal.
`'programa' is up to date.`	Mesmo caso, para um alvo que não é `all`.
`Circular dependency dropped`	Um alvo depende dele mesmo, direta ou indiretamente.
`recipe for target 'x' failed`	O comando da receita retornou erro; leia a mensagem do compilador acima dela.
`undefined reference to ...`	Falta linkar um `.o` ou biblioteca, ou `-l` está antes dos objetos.
`multiple definition of ...`	Função definida em dois `.c`, ou função definida em um header incluído em vários arquivos.
Variável do shell vazia	Em receitas, escreva `$$VAR` (o Make consome um `$`). Ex.: `for f in *.c; do echo $$f; done`.
Espaço sobrando no valor	`X = a # comentario` guarda os espaços antes do `#` no valor. Comente em linha separada.
`cd` não funciona na linha seguinte	Cada linha da receita roda em um shell **diferente**. Use `cd dir && comando` na mesma linha.

12.8 Alvos e variáveis especiais úteis

Item	Função
`.DEFAULT_GOAL := all`	Define explicitamente o alvo padrão.
`.DELETE_ON_ERROR:`	Apaga o alvo se a receita falhar no meio (evita `.o` corrompido).
`.SILENT:`	Silencia todas as receitas (alternativa ao `@`).
`.SECONDARY:`	Impede a remoção de arquivos intermediários.
`$(MAKE)`	Use em vez de `make` ao chamar o make recursivamente; propaga flags como `-j`.
`override VAR = x`	Impede que a linha de comando sobrescreva a variável.

12.9 Dependência de uma biblioteca (libft) e make recursivo

Quando o projeto usa a libft, o Makefile principal precisa construir a biblioteca primeiro e linkar com ela. Isso é feito chamando o Makefile da libft com `$(MAKE) -C libft`. Como o Makefile principal não sabe quando os fontes da libft mudaram, usamos o idioma `FORCE`: a regra sempre é consultada, mas o `ar` da libft só roda se algo mudou, então o executável **não** é religado sem necessidade.

12.10 Projeto 4 — Makefile completo no padrão 42

```
projeto/
├── Makefile
├── includes/
│   └── app.h
├── libft/            (com o seu próprio Makefile)
└── src/
    ├── main.c
    ├── parser/parse.c
    └── exec/run.c
```

Makefile final

```
NAME      := app

CC        := cc
CFLAGS    := -Wall -Wextra -Werror
CPPFLAGS  := -I includes -I libft

SRC_DIR   := src
OBJ_DIR   := obj
LIBFT_DIR := libft
LIBFT     := $(LIBFT_DIR)/libft.a

SRC       := main.c \
             parser/parse.c \
             exec/run.c
OBJ       := $(addprefix $(OBJ_DIR)/, $(SRC:.c=.o))
DEP       := $(OBJ:.o=.d)

ifdef DEBUG
CFLAGS += -g3 -fsanitize=address
endif

ifeq ($(V),1)
Q :=
else
Q := @
endif

GREEN := \033[0;32m
RESET := \033[0m
```

```
all: $(NAME)

$(NAME): $(LIBFT) $(OBJ)
    $(Q)$(CC) $(CFLAGS) $(OBJ) -L$(LIBFT_DIR) -lft -o $@
    @printf "$(GREEN)[OK]$(RESET) %s\n" $(NAME)

$(LIBFT): FORCE
    $(Q)$(MAKE) -C $(LIBFT_DIR) --no-print-directory

$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c
    @mkdir -p $(@D)
    $(Q)$(CC) $(CFLAGS) $(CPPFLAGS) -MMD -MP -c $< -o $@

clean:
    $(Q)$(MAKE) -C $(LIBFT_DIR) clean --no-print-directory
    $(Q)rm -rf $(OBJ_DIR)

fclean: clean
    $(Q)$(MAKE) -C $(LIBFT_DIR) fclean --no-print-directory
    $(Q)rm -f $(NAME)

re: fclean all

FORCE:

.PHONY: all clean fclean re FORCE

-include $(DEP)
```

Por que cada decisão foi tomada

- `all` é o primeiro alvo; `-include $(DEP)` fica no fim, para não virar alvo padrão.
- Fontes listados **explicitamente** (sem `wildcard`), como a 42 exige.
- `-MMD -MP` mantém os headers sob controle sem lista manual.
- `$(OBJ_DIR)/%.o` + `mkdir -p $(@D)` suporta subpastas em `src/`.
- `$(LIBFT): FORCE` constrói a libft sem forçar relink do executável.
- `$(MAKE) -C` propaga flags e funciona com `-j`.
- `Q`/`V=1` e `DEBUG=1` dão controle sem editar o arquivo.

12.11 Checklist de avaliação da 42

- Flags `-Wall -Wextra -Werror` presentes, compilador `cc`.
- Regras obrigatórias: `$(NAME)`, `all`, `clean`, `fclean`, `re` (e `bonus` quando pedido).
- `make` duas vezes seguidas **não** recompila nem relinka.
- `make bonus` não relinka o que já está atualizado.
- Todos os alvos não-arquivo em `.PHONY`.
- Sem `wildcard` nem `*.c`; fontes listados um a um.
- O projeto compila a partir de um clone limpo, sem arquivos gerados no repositório (`.o`, `.d`, executáveis).
- Se o projeto usa a libft, o Makefile a constrói automaticamente.
- Nenhum erro ou aviso ao compilar com `make re`.

> **NOTA**
> Os requisitos exatos variam por projeto. Leia o subject: ele manda mais que qualquer checklist genérico.

Gabaritos dos exercícios

Use esta seção só **depois** de tentar. As respostas mostram uma solução possível; o importante é entender o porquê.

Módulos 1–5 — Os cinco exercícios de fixação

Solução básica

```
NAME   = programa
CC     = cc
CFLAGS = -Wall -Wextra -Werror

$(NAME): main.c
    $(CC) $(CFLAGS) main.c -o $(NAME)

executar: $(NAME)
    ./$(NAME)

clean:
    rm -f $(NAME)

.PHONY: clean executar
```

O que conferir

- **Item 1 e 3:** `$(CC) $(CFLAGS) main.c -o $(NAME)` usa as duas variáveis.
- **Item 2:** `$(NAME)` aparece na regra e na receita, nunca o texto `programa` repetido.
- **Item 4:** `clean` usa `rm -f` (não falha se o arquivo não existir) e está em `.PHONY`.
- **Item 5:** `executar` depende de `$(NAME)` e usa `./$(NAME)`; também precisa de `.PHONY`, pois não gera arquivo `executar`.
- **Erro típico:** esquecer o TAB ou colocar `executar` antes do alvo de compilação, tornando-o o alvo padrão.

Como um desenvolvedor experiente escreveria

```
NAME   = programa
CC     = cc
CFLAGS = -Wall -Wextra -Werror

SRC = main.c
OBJ = $(SRC:.c=.o)

all: $(NAME)

$(NAME): $(OBJ)
    $(CC) $(OBJ) -o $@

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

executar: all
    ./$(NAME)

clean:
    rm -f $(OBJ)

fclean: clean
    rm -f $(NAME)

re: fclean all

.PHONY: all clean fclean re executar
```

Diferenças: objetos separados (recompila só o que mudou), regra de padrão, alvos padrão `all`/`fclean`/`re` e `$@` no lugar de repetir o nome.

Módulos 2–4

- **M2-1:** `ola:` seguido de TAB e `cc main.c -o ola`.
- **M2-3:** o Make mostra `*** missing separator. Stop.`
- **M3-1:** sem `.PHONY`, `make limpar` diz `'limpar' is up to date.` porque existe um arquivo com esse nome.
- **M3-3:** `executar: programa` e o alvo `programa: main.c`.
- **M4-3:** `$(info CFLAGS=$(CFLAGS))` em qualquer linha fora de uma receita.

Módulos 6–8

- **M6-3:** sem o header nas dependências, editar `matematica.h` não causa recompilação (bug clássico).
- **M7-3:** `.PHONY: $(NAME)` faz o Make sempre considerar o alvo desatualizado, então ele religa a cada `make`.
- **M8-3:** o linker lê da esquerda para a direita e só leva da biblioteca o que já foi pedido; se `-lft` vem antes dos objetos, nada foi pedido ainda.
- **M8-2:** `ar t` lista os `.o`; `nm` mostra símbolos (`T` = função definida).

Módulos 9–11

```
# M9-1 (regra estática)
$(OBJ): %.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

# M10-1 (duas formas)
OBJ = $(addprefix obj/, $(SRC:.c=.o))
OBJ = $(patsubst %.c,obj/%.o,$(SRC))

# M10-2
INCS = $(foreach d,a b c,-I$(d))

# M10-3
SRC := $(filter-out main.c,$(SRC))
```

- **M11-3:** sem `-include $(DEP)` o Make volta a ignorar os headers; editar um `.h` deixa de recompilar os `.c` que o usam.

Apêndice — Cola rápida

Estrutura de uma regra

```
alvo: dependencia1 dependencia2 | order-only
    receita (TAB)
```

Variáveis automáticas

Variável	Valor
`$@`	alvo
`$<`	primeira dependência
`$^`	todas as dependências
`$?`	dependências mais novas que o alvo
`$*`	stem do padrão `%`
`$(@D)` `$(@F)`	pasta e nome do alvo

Atribuições

Operador	Efeito
`=`	recursiva (avalia no uso)
`:=`	imediata
`?=`	só se indefinida
`+=`	acrescenta

Compilação em C

Comando	Faz
`cc -c a.c -o a.o`	compila sem linkar
`cc a.o b.o -o prog`	linka objetos
`ar rcs lib.a a.o b.o`	cria biblioteca estática
`cc main.c -L. -lft`	linka com `libft.a`
`-I dir`	pasta de headers
`-MMD -MP`	gera `.d` de dependências
`-g3 -fsanitize=address`	debug e detector de erros de memória

Template mínimo recomendado

```
NAME     := programa
CC       := cc
CFLAGS   := -Wall -Wextra -Werror
CPPFLAGS := -I includes

SRC_DIR := src
OBJ_DIR := obj
SRC     := main.c util.c
```

```
OBJ     := $(addprefix $(OBJ_DIR)/, $(SRC:.c=.o))
DEP     := $(OBJ:.o=.d)

all: $(NAME)

$(NAME): $(OBJ)
    $(CC) $(CFLAGS) $(OBJ) -o $@

$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c
    @mkdir -p $(@D)
    $(CC) $(CFLAGS) $(CPPFLAGS) -MMD -MP -c $< -o $@

clean:
    rm -rf $(OBJ_DIR)

fclean: clean
    rm -f $(NAME)

re: fclean all

.PHONY: all clean fclean re

-include $(DEP)
```

DICA

Parabéns! Se você consegue explicar cada linha desse template sem consultar nada, já escreve Makefiles profissionais sem copiar modelos da internet.